

Round Robin vs SJF Comparison Project

1. Project Overview

This project implements and compares two CPU scheduling algorithms: Round Robin (RR) and Shortest Job First (SJF - Non-preemptive). The goal is to analyse their behaviour under different workloads and evaluate their performance using standard scheduling metrics.

The comparison focuses on:

- Waiting Time (WT)
- Turnaround Time (TAT)
- Response Time (RT)
- Fairness
- Efficiency

2. System Description

The system is a GUI-based CPU Scheduling Simulator developed using Java Swing. It allows users to:

- Input processes dynamically (Arrival Time, Burst Time)
- Specify a time quantum for Round Robin
- Visualize execution using Gantt Charts
- View calculated metrics
- Compare algorithm performance

3. Implementation Details

3.1 Algorithms

Round Robin (RR)

- Uses a time quantum to divide CPU time.
- Processes are executed in a circular queue.
- Ensures fairness by giving each process equal CPU access.

Shortest Job First (SJF - Non-preemptive)

- Selects the process with the smallest burst time.
- Once a process starts, it runs until completion.
- Optimized for minimizing waiting time.

3.2 Metrics Calculation

For each process:

- $\text{Waiting Time (WT)} = \text{Turnaround Time} - \text{Burst Time}$
- $\text{Turnaround Time (TAT)} = \text{Completion Time} - \text{Arrival Time}$
- $\text{Response Time (RT)} = \text{First Start Time} - \text{Arrival Time}$

Averages are calculated for all processes.

3.3 Visualization

- Gantt Charts are used to represent execution order.
- Grid lines and time markers improve readability.
- Each process is assigned a unique colour.
- Ready Queue visualization shows process order over time.

4. Test Scenarios

Scenario A: Basic Mixed Workload

A mix of short and medium processes.

Scenario B: Short-Job Heavy Case

Mostly short processes to highlight SJF efficiency.

Scenario C: Fairness Case

Processes with similar burst times to demonstrate fairness.

Scenario D: Long-Job Sensitivity Case

One long process competing with short ones.

Scenario E: Validation Case

Invalid inputs such as negative burst time or duplicate IDs.

5. Results and Observations

5.1 Round Robin

- Lower response time
- Fair distribution of CPU
- Handles long processes better
- Performance depends on quantum

5.2 SJF (Non-preemptive)

- Lower waiting and turnaround time (in ideal cases)
- Strong preference for short jobs
- Can suffer from convoy effect
- May delay processes significantly

6. Comparison Analysis

Fairness vs Efficiency

- RR is fair but less efficient
- SJF is efficient but less fair

CPU Distribution

- RR distributes CPU evenly
- SJF prioritizes shortest job

Short Job Handling

- SJF executes short jobs faster
- RR may delay them slightly

Quantum Effect

- Small quantum → better responsiveness, more overhead
- Large quantum → less fairness, behaves like FCFS

Response Time

- RR provides faster initial response
- SJF may delay processes depending on order

7. Assumptions

- All processes are CPU-bound.
- Context switching time is ignored.
- SJF is implemented as non-preemptive.
- Time quantum is constant throughout execution.

8. Limitations

- Does not simulate real OS overhead (context switching cost).
- SJF cannot adapt dynamically to new shorter jobs once execution starts.
- No multi-core support.
- Ready Queue visualization is an approximation.

9. Conclusion

Round Robin is more suitable for systems that require fairness and responsiveness, such as interactive environments. It ensures that all processes get CPU access regularly.

SJF is more efficient in terms of minimizing waiting and turnaround time, especially in workloads with many short processes. However, it may lead to poor performance in scenarios where long processes arrive first.

The choice between the two algorithms depends on system requirements:

- Use Round Robin for fairness and responsiveness.
- Use SJF for efficiency when workload characteristics are known.

10. Screenshots



